分形持续学习之前

递归结构生长与长程评估的实现审计

HUSHFUSION Forge 研究

2026 年 10 月

“你的数学将由五个不同的委员会严格核查；你将被要求在联合而无情的攻击面前为它辩护。两年之后，你的成果将被再次审查……因为一件看似完美的成果，其谬误往往要经过数月或数年的潜伏期才会暴露。有时，正是发现者本人发觉了缺陷。”

——阿西莫夫《第二基地》，*“谢顿计划” (1953)*

摘要

持续学习通常被表述为固定计算图下的参数适应; 计算图本身是否应当生长——结构可塑性——是一个独立的问题, 常被断言而鲜被审计。我们构造了一个递归分形环学习器, 通过归一化凸门控混合根节点与后裔节点的输出, 并以相对衰减检测器在改进减缓时生长后裔; 在一个重采样输入的控制环境中, 我们让它与固定单节点学习器配对运行 72,335 轮。审稿触发的实现审计揭示: 在这轮运行中, 后裔节点从未被训练——它们不出现于任何优化器的参数表, 所宣称的深度相关稀疏调度也从未进入任何调度判断。树在未让新组件学习的情况下生长, 因此表面上的边界结果——“无结构优势”——并不是对分形学习器的检验, 而是对门控未训练随机子网的根节点的检验。两个根节点的长程适应本身得到确认 (warm 代价从约 130 步降至约 9 步, 随机基线约 32 步), 生长也是可操作的 (6 次事件; 1 到 7 节点, 封顶), 但结构优势在这轮运行中不可判定。我们报告审计、修复后的实现, 以及决定性重评的条件。生长事件不是优势的证据; 可复现性始于优化器的参数表。

1 引言

持续学习通常被表述为一个权重更新问题: 保持图结构固定, 调整参数, 使新任务被学习而不遗忘旧任务 [1, 3, 2]。这个框架留下了一个未被提出的问题: 当任务本身变化时, 应该改变的不仅是权重, 还有更新环路的组织吗? 这个问题对长程控制器至关重要——学习器必须在数万轮中跟踪移动的最优解, 同时保持稳定的响应。然而, 结构生长是否有益, 几乎总是被断言, 而从未被测量。

我们把它当作一个测量问题, 而随后的审计就是本文的贡献。架构是一个递归分形环学习器: 密集根节点、通过归一化凸门控混合的稀疏后裔、以及在改进率衰减时生长后裔的相对衰减检测器。主实验在重采样输入的控制环境中, 让该学习器与固定单节点学习器配对运行 72,335 轮。评审后的代码审计发现了主实验实现中的一个决定性缺陷: 后裔节点从未加入任何优化器的参数表, 所宣称

表 1: 长程配对结果 (72,335 轮)。warm 行为推荐器, cold 行为随机对照。越界 = 越过安全上限 μ_{ceil}。审计警示: 本轮分形后裔从未被训练 (§3.4), 分形列反映的是门控未训练子网的根, 而非经训练的分形学习器。

指标	固定	分形
warm 均值步数 (全程)	11.1	11.5
warm 均值步数 (收敛段, 轮 ≥2000)	∼9	∼9
cold 均值步数	31.6	31.9
越界率	5.9%	5.8%
最终树节点	1	7

的深度缩放更新调度也从未进入调度决策。因此, 长程运行评估的是一个门控未训练随机子网的根节点, 而非分形学习器。我们报告仍然成立的部分 (长程适应是真实的; 生长是可操作的)、被撤回的部分 ("无结构优势" 的主张在这轮运行中不可判定), 以及决定性比较所需的修复与验证计划。我们发布审计线索与修复后的代码。

2 结果

2.1 两个根学习器的长程持续适应均成功

固定单节点网络与递归生长的分形树, 在每个回合以同一重采样初始条件 μ_0 配对 (协议见方法)。每个学习器基于自身累积的经验选择控制点并执行环境。审计确定的是: 在这轮运行中, "分形" 学习器的后裔未被训练; 可观察行为是门控未训练随机子网的根节点。在此警示下, 这轮运行仍然验证了根学习器的长程适应。

两个根都学习了并持续学习。warm 性能在前 1,000 轮从约 130 步降至大约 2,000 轮后的约 9 步, 并在剩余 70,000 轮中保持; cold 随机基线全程保持约 32 步 (表 1)。持续适应因此有效且稳定, 没有衰减。此结果独立于后裔训练缺陷: 它是根学习器的性质, 两个根都表现出它。

2.2 结构生长至封顶

分形树生长了六次 (第 46、57、73、80、87、94 轮), 从 1 节点到 7 节点, 然后停止。生长由相对衰减检测器驱动 (方法): 当验证损失的指数加权改进率低于该结构早期改进率的某个比例时, 生长后裔。达到七节点后, 检测器在剩余 72,000 轮中从未再触发。我们不将其归因于自主饱和: 七是实现中的硬安全上限 (`grow_check` 在 `n_nodes() >= 7` 时返回 False), 因此生长是被封顶停止的, 而非涌现的均衡。没有封顶时树是否会继续生长, 未被检验。

审计使这幅图景复杂化。生长是可操作的——检测器触发, 图结构发生突变——但生长的后裔未被训练 (§3.4)。因此, 这轮运行展示的是检测器与图突变在长程上工作, 而非生长的组件会学习。生长事件是真实的; 它们的学习贡献从未被发挥。

表 2: 什么成立、什么撤回、什么不可判定。此裁决框是本文的完整主张集。

成立	撤回	不可判定
根节点长程适应: warm 130→9 步 vs. 随机 ∼32, 保持 72,000 轮	“无结构优势”(原声称的边界结果)	经训练分形学习器在配平计算量下的结构优势
相对衰减生长检测器: 6 次事件, 1→7 节点 (封顶停止)	稀疏 D^{-d} 更新调度作为已执行机制	没有 7 节点封顶时生长是否会饱和
控制环境作为确定性基准	后裔训练作为已执行机制	修复、播种、配平计算量重跑的结果

2.3 结构优势不可判定: 实现审计

本文早先版本声称一个边界结果: “无结构优势”。评审后对主实验实现的审计撤回这一主张。三个发现是决定性的。

后裔从未被训练。在 `design_daemon_fractal.py`(被审版本: commit `10ba961^`; 仓库 HEAD 现为修复代码) 中, 每个节点的优化器只对其自身网络与门控参数构造 (“opt = Adam(list(self.net.parameters()) + [self.gate])”)。生长节点的子节点被追加到 `self.children`, 但从未加入任何父节点的优化器参数表, 主循环只对根调用 `train_step`。因此后裔的参数从不接收梯度步。每个子节点唯一的操作是 warm-up 计数器递减。所生长的 “树” 是一个门控固定、随机初始化子网的根网络。此处引用的审计行号对应 `10ba961^`; 检出 HEAD 的读者会看到修复后的循环, 因此被审快照以其 commit 哈希钉定。

所宣称的稀疏调度从未执行。代码计算 `update_freq = D**depth` 并存储, 但没有任何调度决策查询它。深度相关的更新频率是文档, 而非行为。

评估不能按所述复现。 72,335 轮 trace 及其分析脚本不在仓库中; daemon 以墙钟时间为其 RNG 播种 (`int(time.time()) % 10000`); 重跑不复现报道的数字。仓库的 `README` 承认该 trace 为 “活数据, 未提交”。

后果。配对长程差 (11.5 vs. 11.1 warm 步) 与三 seed、701 条记录比较, 比较的都是一个经训练的根对一个门控未训练子网的根——这一条件既不确认也不否认分形优势。正确的表述是**这轮运行不可判定**。长程适应与生长检测仍然成立; 结构优势需要一个修复的实现与一次决定性的重跑。

2.4 审计未触及的部分

三个早期结果独立于后裔训练缺陷, 仍然成立。(i) 长程持续适应是真实的: 两个根从约 130 收敛到约 9 warm 步, 对照随机基线约 32, 保持 72,000 轮。(ii) 相对衰减检测器可操作: 6 次生长事件, 1 到 7 节点, 然后被封顶停止。(iii) 控制环境是带安全上限的合理确定性基准; 这不受学习器实现影响。此三者成立; 结构优势的主张撤销。

3 相关工作

我们的设计处于三类思想的交汇点: 基于优化的元学习——将快速任务适应与较慢的元参数更新分离 [1, 3]; 持续学习整合——处理序列任务下的遗忘 [2]; 以及分形分配论证——当边际收益随深度递减时, 计算不必在层级间均匀分布, 此原则可追溯至默里对分支网络最小功的分析 [6], 并在我们配套的 Lean 开发中形式化 [5]。我们将其作为动机而非预测。本文不声称解决分形组织是否有益的问题; 其贡献在于展示一次看似干净的长程比较如何静默地未能检验假设, 以及一次决定性检验需要什么。

4 讨论

两句话必须分开, 而审计强化了这一区分。“递归结构可塑性可实现且稳定” 现在被限定: 生长事件触发, 图结构突变, 但在被审运行中生长的组件从未学习, 所以我们只能说检测器与突变工作, 而非生长的学习器。“结构生长改善性能” 是本文早先的主张, 现在被撤回: 这轮运行没有检验一个经训练的分形学习器, 因此其零结果是不可判定的。混淆 “树生长了” 与 “生长的组件学习了” 正是本次审计纠正的错误; 将两者分开是本文的方法论要点。

优势能否出现的问题仍然开放。分形分配论证 [5] 要求边际收益足够强地递减, 使得将预算集中于早期层级胜过平摊。在数据有限的小学习器上, 边际收益可能不会急剧下降; 在专家之间分割预算可能只会稀释每位专家的证据。这些是假设, 而非发现: 被审运行无法对它们发言, 因为它从未训练分形学习器。

两条评审意见锐化了决定性检验。第一, 计算量必须配平: 七节点的分形树每轮消耗的梯度步数是单节点的数倍, 因此朴素重跑会把结构与计算量混淆。修复后的实现逐轮计数树中的总梯度步数, 并给固定学习器相同预算 (Forge commit `53e4938`)。第二, 等价界必须在重跑开始前预先注册, 而非事后选择: 我们将等价区间固定为 ± 1 步 (度量的整数步分辨率)、收敛窗口固定为轮 ≥ 2000、最小 5,000 与最大 10,000 warm 轮 (在 5,000 处一次窥视, 若区间仍跨界则以 “未决” 停止, 防止持续窥视), 以配对 bootstrap 置信区间为判定规则 (细节见 `PREREGISTRATION.md`)。遵循这些规则的重跑, 是边界变得可判定而非又一次断言的唯一途径。

方法论的教训是程序性的。分三步评估结构生长:(i) 图是否能安全突变,(ii) 突变后学习器是否持续适应,(iii) 突变是否在配平计算量与数据下改善预先注册的指标。本文跑了第 (i) 与 (ii) 步——而审计显示第 (ii) 步仅部分满足 (图突变了, 但生长的组件没有训练)。第 (iii) 步未达, 因为实现缺陷使其不可判定。那一步是正确的下一个实验。

5 方法

5.1 递归分形环架构

架构定义如下; §3.4 审计了部署实现实际做了什么。每个节点 v 持有一个预测器 $f_v(x;\theta_v)$, 并可有后裔 $\mathcal{C}(v)$。节点输出为凸组合,

$$y_v(x) = g_{v,0}\, f_v(x;\theta_v) + \sum_{u\in\mathcal{C}(v)} g_{v,u}\, y_u(x), \qquad g_{v,j}\ge 0, \quad \sum_j g_{v,j} = 1, \tag{1}$$

门控由 softmax 产生。后裔不会立即被信任: 它们在进入父节点混合之前被独立预热, 这防止了 "死子节点" 失效——零初始化门控给予子节点零贡献, 因此零梯度。

根节点每轮更新; 深度 d 的节点以 D^{-d} 的频率更新, 稀疏因子 D(我们用 $D=4$)。这把计算集中于浅层前缀——分形分配规则的操作类比。如审计所示, 该稀疏调度在部署代码中被指定但未被执行。

5.2 生长规则

生长由验证损失的相对衰减触发。设 $\bar{\imath}_t$ 为指数加权改进率, $\imath_0$ 为该结构早期平均改进率。当 $\bar{\imath}_t/\imath_0$ 连续多次观测低于阈值 (部署运行中为 0.25) 时, 生长一个后裔; 检测器在生长后重置。干净损失过滤器排除安全惩罚值, 因为惩罚尖峰不是结构瓶颈的证据。我们包含促成设计的失败: 最初的绝对规则 ("损失不再下降") 从不触发, 因为浅层模型可以以缓慢速率永远改进。

5.3 控制环境

我们由双流环模型装配一个确定性 μ-动力学控制器:

$$\mu_{t+1} = \mu_t + \eta_t(1-\mu_t), \qquad \eta_t = \eta_{\text{ext}} + (2\lambda - \lambda^2)(1-\mu_t), \tag{2}$$

其中 η_{ext} 是外部驱动, λ 是候选自展平项的收敛参数。任务是达到 $\mu \ge 0.999$(工作窗口) 而不越过 $\mu_{\text{ceil}} = 1 - m_e/m_i \approx 0.999780568$(安全上限)。代价是到达窗口的步数; 越界是失败。环境是确定性的, 有清晰的安全边界, 并且是重采样的: 每轮抽取新的 $\mu_0 \sim U(0.01, 0.8)$。我们将其描述为重采样而非漂移: 从控制点到代价的映射是固定的, 因此学习器面对的是输入重采样而非变化的任务。它是方法研究的基准, 而非任何实验性反引力装置的证据; λ 在上游模型中仍是输入缺口。

5.4 配对长程协议

每轮两个学习器看到相同的 μ_0。每个从自身经验选择控制点 ($(\eta_{\text{ext}}, \lambda)$ 的 20×10 网格上的 UCB), 执行环境, 并从最近经验滑动窗 (容量 400) 更新参数。奇轮为 cold 随机对照, 偶轮为 warm 推荐。部署运行使用 72,335 轮、每 10 轮独立检查点、以及失败时重启 daemon 的监督器。分形树允许无预设深度生长, 受 7 节点安全上限约束。如 §3.4 所述, 部署实现未将后裔参数纳入任何优化器, 因此此处的协议描述是预期设计, 而非执行行为。

5.5 塑造设计的失败分析

四个失败值得记录。(i) 绝对 “无改进” 生长检测器在浅层网络上从不触发 (它们缓慢地永远改进); 相对衰减检测器修复了它。(ii) 零初始化的子门控产生死子节点; 均匀门控初始化加预热阶段修复了它。(iii) 生长一层而不重新初始化逐参数步长元参数, 损害了快速适应环, 因此结构生长与元参数生长不能视为独立操作。(iv) 标准 RSI 指标不适用于单调改进的损失轨迹 (单侧增量使 RSI $\equiv$ 100), 因此此处该词用于 “递归自我改进”, 而非金融指标。

数据与代码可用性

被审实现、修复与分析脚本在 HUSHFUSION Forge 仓库中 [4]。被审快照为 commit `10ba961^`(子节点未训练); 修复后的配平计算量版本为 commit `53e4938`。促使审计的 72,335 轮 trace 是研究服务器上的活数据, 未提交。决定性重跑已在 `PREREGISTRATION.md` 中预注册 (等价界 ± 1 步、收敛窗口 $\geq$2000、$\geq$5000 warm 轮、配对 bootstrap CI), 并以 seed 20261012 在配平计算量下启动; *截至本文写作, 重跑仍在进行, 其结果尚未报告*——本文的主张基于审计与表 2 所列的存活结果, 而非重跑。写作标准见 `WRITING-STANDARD.md`。

参考文献

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. *arXiv preprint arXiv:1707.09835*, 2017.

[4] Yuanjie Liu. HUSHFUSION Forge: design search, knowledge reuse, and continual-learning daemons. `https://github.com/logos-42/hushfusion-forge`, 2026. Long-horizon fracta-loop trace and reproduce scripts.

[5] Yuanjie Liu. LmPrinciple: Fractal allocation theorems for diminishing-return structures. `https://github.com/logos-42/lm-principle`, 2026. Lean formalization: Fractal.lean, prefix_allocation_optimal, fractal_beats_uniform.

[6] Cecil D. Murray. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *Proceedings of the National Academy of Sciences*, 12(3):207–214, 1926.